

The LLM API: batch strings to fixed sampling servers

front: what to ask, what to design · Paul's version of SD Q1 · size-or-time is the whole question

Anthropic question this is: SD Q1 inference API, the fixed-batch dispatcher variant (1point3acres card: `batchstring(inputs) → outputs`, 1–100 strings, one output per input in order, ~100 ms flat, one batch per server; hellointerview "Batched Model Runs", 9 staff reports). **Paul Grunchich runs this prompt himself** ("as strings come in, optimize your usage of the GPUs; people overthink it: it is the batching algorithm plus scale and capacity"). His numbers: 20k req/s, non-streaming, 500 ms, servers finite and error when busy. **Same family:** the 8-GPU two-model follow-up, the doc-review round (its planted flaws are this design done wrong), hack2hire GPU inference serving. **Paul's own follow-up:** durations 1 s to 1 h, batch time = longest, 15% overhead → bin pack by duration bucket.

1 REQUIREMENTS 0–5 min · units first: strings, not tokens

- **Ask:** is 100 ms independent of string length and batch size? one batch per server at a time? servers cannot be added, and busy = error? streaming (no)? 500 ms p50 or p99? tiers, and may we reject? client retries and idempotency?
- **A client can:** send a string and get its completion in one response within 500 ms, or a clear rejection (429 retry-after, 504 past deadline).
- **Technical challenges:** fixed capacity → the only levers are batch fill and admission · 100 ms fixed cost → timer is the latency lever at low load, size cap the throughput lever at high · sync request → async batch · shed early by tier, never let the queue grow · a server error is a "no free slot" signal.
- **Anchors:** 100 strings / 0.1 s = 1,000 req/s per server · 20k → 20 servers floor, ~26–28 with headroom · budget 500 = ~50 wait + 100 GPU + net + reserve.

2 CORE ENTITIES 5–8 min · the batch map is the recovery record

- **Request**(id, tenant, tier, string, arrival_ts, deadline = arrival + 500 ms, future)
- **Batch**(id, server, strings[≤100], positions, dispatched_ts)
- **Server**(id, state free | busy | unhealthy, last_error_ts)
- **TierQueue**(tier, pending[], oldest_ts)
- Not entities: the token, the model version (fixed here), the cache. **Never shard a prompt:** a long prompt is one string in a batch of 1–100.

3 API 8–11 min · one public endpoint, the given contract inside

- `POST /v1/complete {string, tier?} → {completion} · 429 {retry_after} · 504 past deadline · header Idempotency-Key` (a retry attaches to the existing future)
- internal: `batchstring(strings[]) → outputs[]` (1..100, 100 ms, one per server at a time) · `dispatch(batch) → server` with a free slot · `GET /servers health` and free slots

4 DATA FLOW 11–18 min · one request, then the overload case

- arrive → gateway: tier quota; reject early if the tier queue is already past budget → tier queue with a future →
- **batcher** pulls paid first, weighted from free; **flushes at 100 strings or when the oldest waited ~50 ms** →
- **dispatcher** hands the batch to a server with a free slot (pull model) → outputs in order after 100 ms → **mapper** resolves each future by position → response.
- **Deadline sweeper:** a request that will miss 500 ms is dropped with 504 before dispatch; a wasted slot is worse than a rejection. **40k in, 20k out:** queue would grow 20k/s → bound by deadline, 429 at the door, free first.

5 HIGH-LEVEL DESIGN 18–30 min · the board

- Clients → gateway (auth, tier quota, early reject) → tier queues → batcher (size or time) → dispatcher (free-slot list, health) → N sampling servers → response mapper (batch position → future) → client. Beside it: batch map for recovery, metrics (queue depth per tier, fill, wait, drops, errors), deadline sweeper.
- **Paul's two failure modes, say both:** high load, a partial batch wastes 100 ms of a server you cannot replace → hold to 100; low load, waiting for 100 breaks the SLA → flush at ~50 ms. Queue depth tells you which regime you are in.
- **Servers that error when busy:** the dispatcher keeps a free-slot list from ACKs; an error marks the server busy until its next completion; still erroring → unhealthy, its batch re-dispatched from the map if the deadline allows.

6 DEEP DIVES 30–45 min · what was wrong on the day, then the follow-up

- **Validated wrong:** "shard a 1,000-token prompt across GPUs" (a string is one request) · "bin pack by GPU memory" (servers run one batch at a time; the router finds a free slot). **Validated right:** deadline drop, tier queues, fill-or-flush, health checks. **Missing:** the 20-server sum, the sync → async mapping.
- **Sync to async:** handler holds a future; batcher records request → batch → position; results routed by position; retries attach by idempotency key.
- **Tiers:** weighted pulls so free is not starved at moderate load; under overload shed free first by policy, and say so.
- **Paul's follow-up (durations 1 s–1 h, batch time = longest, 15% overhead):** geometric duration buckets (1–1.15 s, ~1.32 s, ...) so the shortest in a batch is within 15% of the longest; one queue per bucket; batch within a bucket only; reserve capacity for long buckets; estimate from string length or history, re-bucket on the fly. **Bin packing by time, not memory.**
- **Bow:** 20 servers is the floor; batches fill to 100 under pressure and flush at 50 ms without it; requests that will miss 500 ms are rejected at the door, free first; a busy error is a free-slot signal; the follow-up is the same batcher with duration buckets.

The board at minute thirty

